

Les moniteurs en Java

Fabrice.Kordon@lip6.fr



Petit supplément sur les objets Java

2

Ils intègrent un système d'exclusion mutuelle!

- Intégré dans la «mère de toutes les classes»

 - ▶ **Object**

- Trois primitives

 - ▶ **wait()**

 - ▶ **notify()**

 - ▶ **notifyAll()**

Proche de la notion de condition dans un Moniteur

- Opération Attendre

 - ▶ **wait()**

- Opération signaler

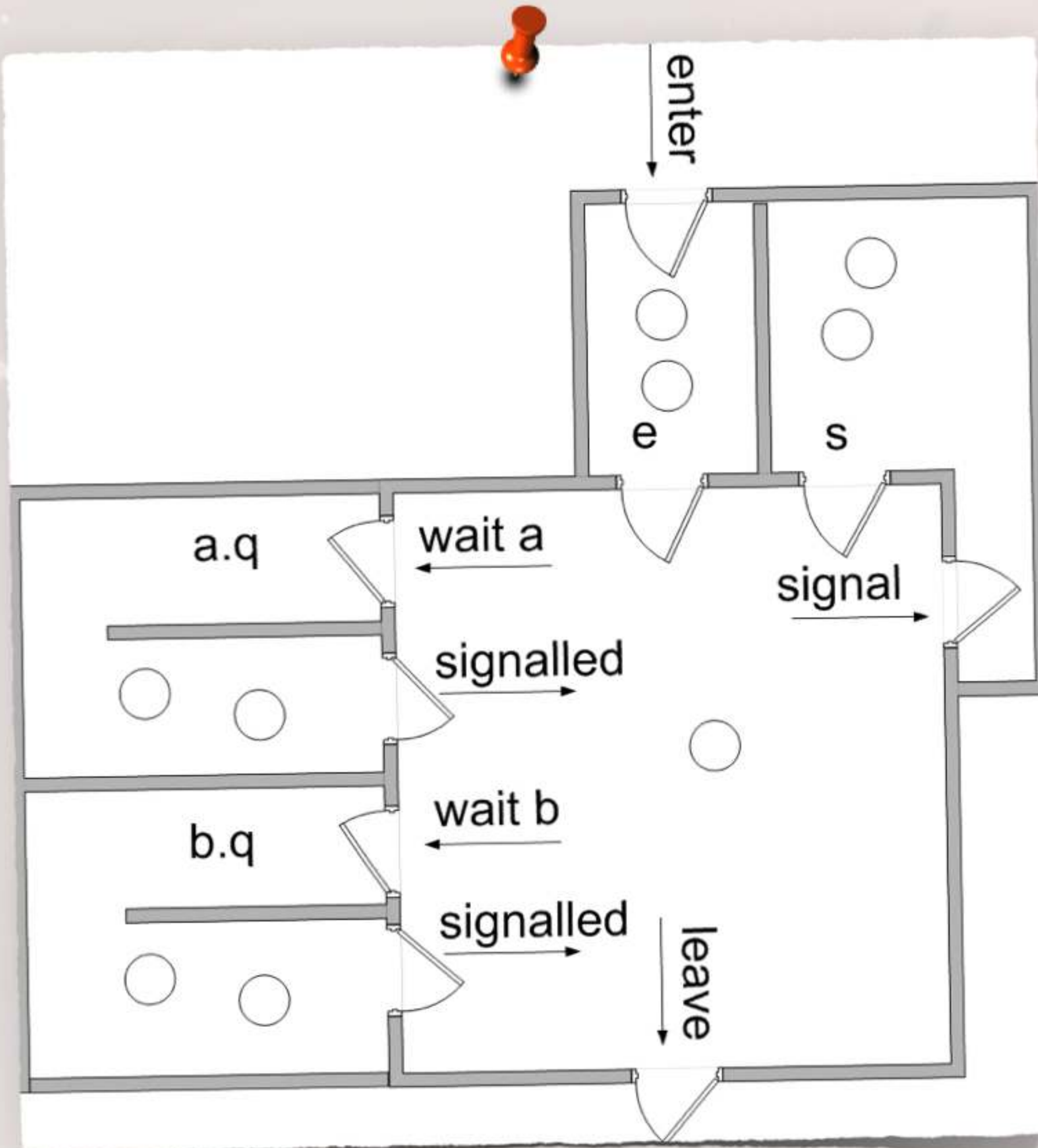
 - ▶ **notify/notifyAll**

- Opération Empty?

 - ▶ **À gérer «à la main»**

Moniteurs de Hoare versus moniteurs associés aux objets Java

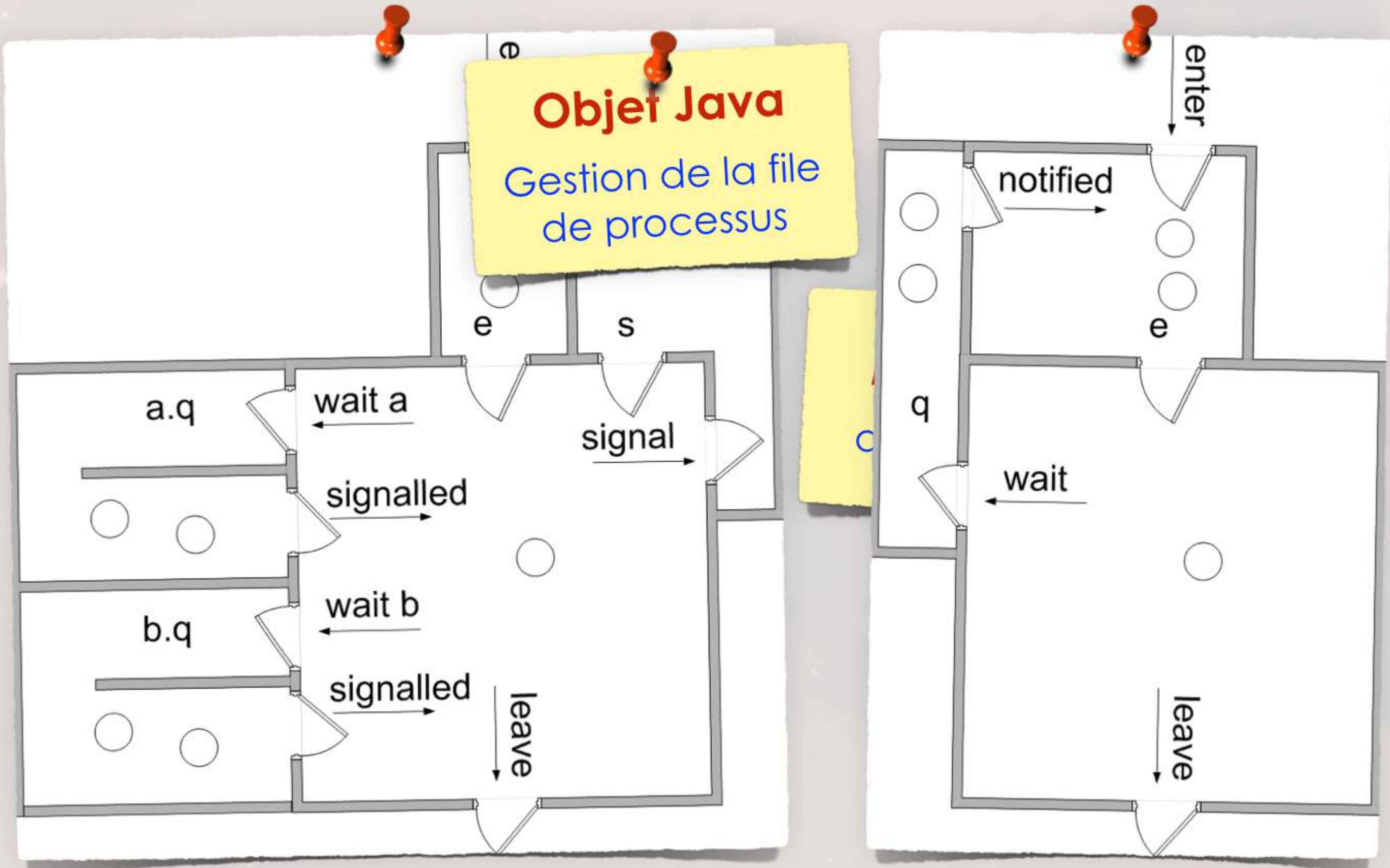
3



Moniteur de Hoare
deux conditions (a et b)

Moniteurs de Hoare versus moniteurs associés aux objets Java

3



Écrivains/Lecteur (la classe Ecrivain)

4

```
import java.util.Random;

class Ecrivain implements Runnable {
    Random generateur = new Random();
    private MoniteurVariable laVariable;

    Ecrivain (MoniteurVariable var) {
        laVariable = var;
    }

    public void run() {
        int retard = generateur.nextInt(1000);
        int valeur = generateur.nextInt(100);

        try {
            Thread.sleep(retard);
        }
        catch (Exception e) {
        }
        System.out.println ("l'Ecrivain démarre après " + retard + " ms");
        System.out.println ("l'Ecrivain va écrire " + valeur);
        laVariable.valeurEcrire = valeur;
        laVariable.finEcrire();
    }
}
```


Écrivains/Lecteur (la classe Lecteur)

5

```
class Lecteur implements Runnable {
    Random generateur = new Random();
    private MoniteurVariable laVariable;

    Lecteur (MoniteurVariable var) {
        laVariable = var;
    }

    public void run() {
        int retard = generateur.nextInt(1000);
        int valeur;
        try {
            Thread.sleep(retard);
            System.out.println ("Le lecteur démarre après " + retard + " ms");
            laVariable.debutLire();
        }
        catch (InterruptedException e) {
            System.err.println ("Gros problème " + e);
        }
        valeur = laVariable.valeurEcrire;
        System.out.println ("Le lecteur a lu " + valeur);
    }
}
```


Écrivains/Lecteur (la classe MoniteurVariable)

6

```
class MoniteurVariable {
    public int valeurEcrire;

    private boolean termine;

    private final Object fini; // doit-être stable ;- )

    MoniteurVariable() {
        fini = new Object();
        termine = false;
    }

    public void finEcrire() {
        synchronized (fini) {
            termine = true;
            fini.notify(); // doit être dans un bloc synchronized
        }
    }

    public void debutLire() throws InterruptedException {
        synchronized (fini) {
            if (! termine) {
                fini.wait(); // doit être dans un bloc synchronized
            }
        }
    }
}
```

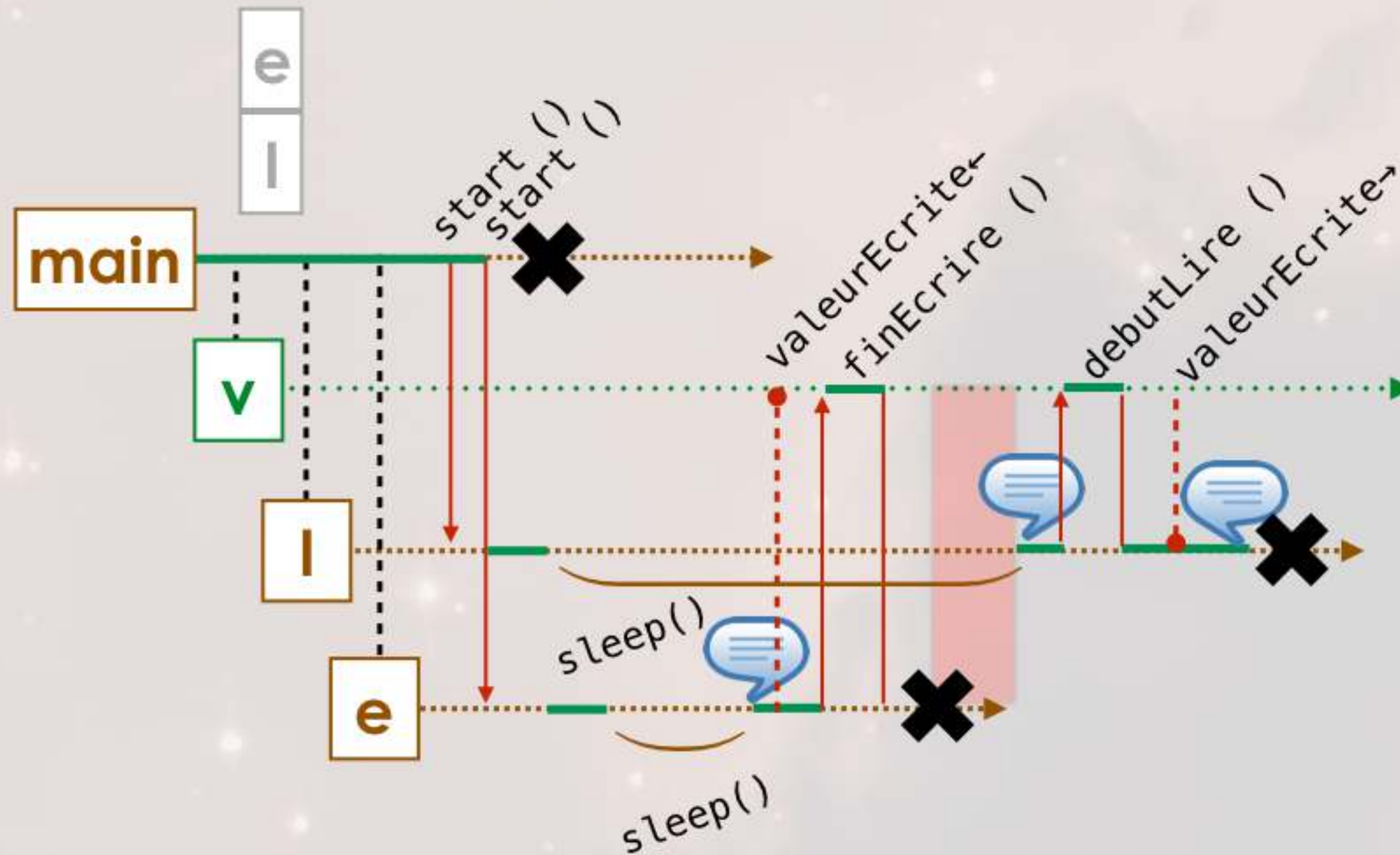

Écrivains/Lecteur (la classe principale)

7

```
public class LecteurEcrivain1 {  
    public static void main(String []args) {  
        MoniteurVariable v = new MoniteurVariable();  
  
        Thread l = new Thread(new Lecteur(v));  
        Thread e = new Thread(new Ecrivain(v));  
  
        l.start();  
        e.start();  
    }  
}
```


Analyse d'une exécution l'écrivain est en avance

8

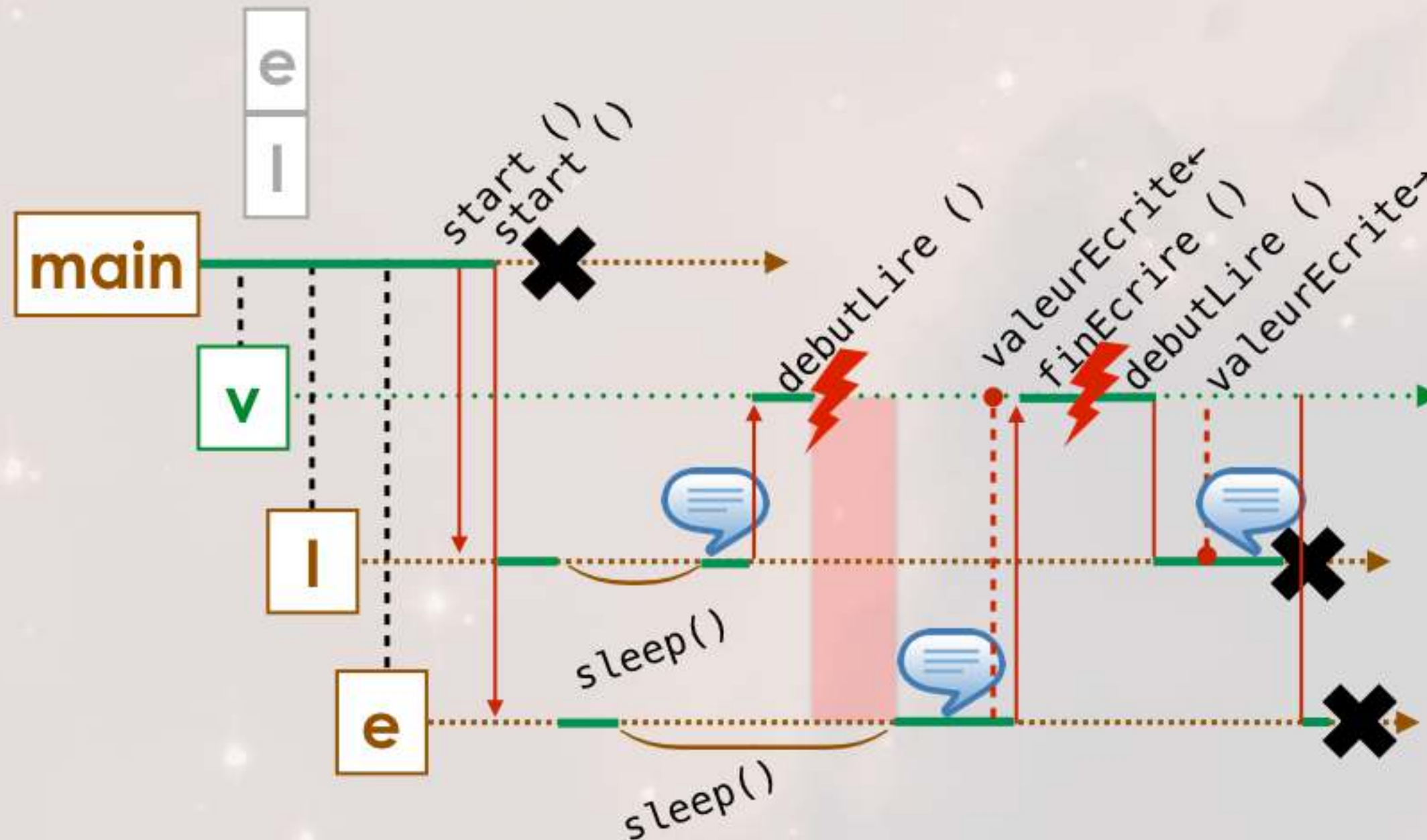


```
$ java LecteurEcrivain1
l'Ecrivain démarre après 458 ms
l'Ecrivain va écrire 0
Le lecteur démarre après 939 ms
Le lecteur a lu 0
```

```
public void debutLire() throws InterruptedException {
    synchronized (fini) {
        if (! termine) {
            fini.wait(); // doit être dans un bloc synchronized
        }
    }
}
```


Analyse d'une exécution le lecteur est en avance

9



```
$ java LecteurEcrivain1
Le lecteur démarre après 441 ms
l'Ecrivain démarre après 884 ms
l'Ecrivain va écrire 54
Le lecteur a lu 54
```

```
public void debutLire() throws InterruptedException {
    synchronized (fini) {
        if (! termine) {
            fini.wait(); // doit être dans un bloc synchronized
        }
    }
}
```


En guise de conclusion...

10

Ce qu'il faut retenir

- Possibilité de suspendre et reprendre un flux avec un moniteur
 - ▶ **Attention, ordre de reprise non garanti**
- En Java, un moniteur est associé à chaque objet
- On peut jouer avec les moniteurs de plusieurs objets

Différences sémantiques par rapport à Hoare

- Pas de suspension systématique sur un `notify()/notifyAll()`
 - ▶ **Pas de priorité sur la reprise d'un thread ayant exécuté `notify()/notifyAll()`**